دانشگاه هرمزگان
دانشکده فنی و مهندسی
گروه مهندسی برق و کامپیوتر

داده‌ساختارها و الگوریتم‌ها

جلسه پانزدهم: آدرس‌دهی باز و درهم‌سازی سراسری

دکتر شهرام گلزاری

پاییز ۱۴۰۵

اهداف جلسه

در پایان این جلسه انتظار داریم که:

- وضعیت جدول با وارسی خطی، وارسی درجه‌ی ۲ یا درهم‌سازی دوگانه را پس از دنباله‌ای از درج، حذف (با نشانه‌ی deleted) و جست‌وجو رسم و تعداد وارسی‌های هر عمل را تعیین کنیم.
- تشخیص دهیم که یک دنباله‌ی وارسی همه‌ی درایه‌های جدول را می‌پیماید یا نه.
- برای خانواده‌ای از توابع درهم‌سازی که به‌صورت جدول داده شده است، تعیین کنیم که سراسری است یا نه.
- با داشتن n و m، کران امید ریاضی طول لیست یک کلید را در درهم‌سازی سراسری با زنجیره‌سازی به دست آوریم.
- با داشتن p، m، a و b مقدار $h_{a,b}(k)$ را حساب کنیم و برای دو کلید، تعداد توابع $\mathcal{H}_{p,m}$ را که آن‌ها را برخورد می‌دهند بشماریم.

یادآوری جلسه‌ی قبل

در جلسه‌ی قبل دیدیم که:

- برخورد اجتناب‌ناپذیر است و در زنجیره‌سازی، همه‌ی کلیدهایی که به یک خانه هش می‌شوند در لیست پیوندی آن خانه قرار می‌گیرند.
- ضریب بار $\alpha = n/m$ میانگین طول زنجیره‌هاست.
- با فرض درهم‌سازی یکنواخت مستقل، جستجوی ناموفق و جستجوی موفق هر دو به‌طور میانگین زمان $\Theta(1+\alpha)$ می‌گیرند.
- در درهم‌سازی یکنواخت مستقل، احتمال برخورد هر دو کلید متمایز حداکثر $1/m$ است.

آدرس‌دهی باز

آدرس‌دهی باز

«در «آدرس‌دهی باز» (open hashing)، یک آرایه به‌اندازه‌ی m به‌عنوان جدول درهم‌سازی استفاده می‌شود که حداکثر m عنصر در آن جای می‌گیرد و در هر درایه، فقط یک عنصر ذخیره می‌شود. بنابراین، برای استفاده از این روش، تعداد بیشینه‌ی عناصر را باید از قبل بدانیم، یا آن‌که از «جدول درهم‌سازی پویا» (dynamic hash table) استفاده کنیم که در آن اندازه‌ی جدول برحسب نیاز، کم یا زیاد (معمولاً نصف یا ۲ برابر) می‌شود.»

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۲۷۸

برخلاف زنجیره‌سازی، در آدرس‌دهی باز هیچ عنصری بیرون از جدول نگهداری نمی‌شود، پس $n \leq m$ و $\alpha \leq ۱$.

«برای درج یک عنصر در آدرس‌دهی باز، باید تعدادی از درایه‌های جدول «وارسی» (probing) شوند تا جای مناسب برای درج یک عنصر پیدا شود. برای این‌کار تابع درهم‌سازی را به این صورت تعریف می‌کنیم:

$$h : U \times \{0, 1, \ldots, m-1\} \to \{0, 1, \ldots, m-1\}.$$

»

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۲۷۸

دنباله‌ی وارسی

«در آدرس‌دهی باز و برای یک کلید k به درایه‌های زیر که به‌صورت متوالی وارسی می‌شوند، «دنباله‌ی وارسی» k (probe sequence) می‌گوییم.

$$\langle h(k, ۰), h(k, ۱), \ldots, h(k, m-۱) \rangle$$

برای جست‌وجوی یک کلید، دنباله‌ی وارسی آن دنبال می‌شود تا آن کلید یافت شود، وگرنه کلید در جدول موجود نیست.»

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۲۷۸

درج در آدرس‌دهی باز

«برای درج، دنباله‌ی وارسی بررسی می‌شود تا اولین درایه‌ی خالی پیدا شود و عنصر مورد نظر در آن‌جا درج می‌شود. اگر چنین جای خالی‌ای پیدا نشد، در آن‌صورت پیام خطای overflow چاپ می‌شود.»

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۲۷۸

«توجه کنید که تابع درهم‌سازی $h(k, i)$ ممکن است طوری طراحی شده باشد که دنباله‌ی وارسی یک کلید، به یک درایه بیش از یک بار برود و در نتیجه، به بعضی از درایه‌ها اصلاً رجوع نکند.»

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۲۷۸

مثال

با $m = ۴$، اگر دنباله‌ی وارسی کلید k برابر $\langle ۲, ۰, ۲, ۰ \rangle$ باشد و درایه‌های ۰ و ۲ پر باشند، درج k با خطای سرریز پایان می‌یابد، هرچند درایه‌های ۱ و ۳ خالی‌اند.

درج و جست‌وجو

```
HASH-INSERT(T, k)
1  i = 0
2  repeat j = h(k, i)
3      if T[j] == NIL
4          T[j] = k
5          return j
6      else i = i + 1
7  until i == m
8  error "hash table overflow"
```

```
HASH-SEARCH(T, k)
1  i = 0
2  repeat j = h(k, i)
3      if T[j] == k
4          return j
5      i = i + 1
6  until i == m
7  return NIL
```

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۲۷۹، با نمادگذاری *CLRS*

HASH-SEARCH در بدترین حالت همه‌ی m درایه‌ی دنباله‌ی وارسی را بررسی می‌کند، حتی اگر به درایه‌ی خالی برسد.

توقف زودتر جست‌وجو

«دقت کنید که اگر به‌جای $i = m$ دستور $\text{until } i = m$ دستور $\text{until } A[j] = \text{null or } i = m$ را قرار دهیم، ظاهراً موجب تسریع عمل درج می‌شود (...) اگر عنصر در جدول نباشد لزومی به گشتن همه‌ی دنباله‌ی وارسی نیست و اولین درایه‌ی خالی جواب منفی را تولید می‌کند. اما در صورت این تغییر، اگر جست‌وجو با حذف همراه شود، چنان‌چه خواهیم دید، ممکن است مشکل ایجاد کند.»

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۲۷۸

این تغییر به رویه‌ی HASH-SEARCH مربوط است: با آن، جست‌وجوی ناموفق با رسیدن به نخستین درایه‌ی خالی پایان می‌یابد.

حذف ساده

«برای حذف یک عنصر، ابتدا آن‌را پیدا می‌کنیم و مقدار آن درایه را برابر تهی قرار می‌دهیم.»

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۲۷۹

```
    HASH-DELETE(T, k)
1   i = HASH-SEARCH(T, k)
2   if i ≠ NIL
3       T[i] = NIL
4   else error "key k not found"
```

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۲۷۹، با نمادگذاری *CLRS*

نشانه‌ی حذف

«اما با انجام تغییرات زیر می‌توانیم کار جست‌وجو را کمی سریع‌تر کنیم. پس از هر عمل حذف، به‌جای این‌که در درایه عنصر حذف‌شده، مقدار null قرار دهیم، مقدار خاص "deleted" را می‌نویسیم. در این صورت، درایه‌ی "deleted" برای جست‌وجو مانند درایه‌ی پر است. اما برای درج، اگر عنصر قبلاً وجود نداشته باشد، این درایه مانند درایه‌ی تهی محسوب می‌شود.»

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۲۷۹

```
HASH-DELETE-NEW(T, k)
1  i = HASH-SEARCH(T, k)
2  if i ≠ NIL
3      T[i] = DELETED
4  else error "key k not found"
```

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۲۸۰، با نمادگذاری *CLRS*

جست‌وجو و درج تازه

```
HASH-INSERT-NEW(T, k)
1  if HASH-SEARCH(T, k) ≠ NIL
2     error "item already exists"
3  i = 0
4  repeat j = h(k, i)
5     if T[j] == NIL or T[j] == DELETED
6        T[j] = k
7        return j
8     else i = i + 1
9  until i == m
10 error "hash table overflow"
```

```
HASH-SEARCH-NEW(T, k)
1  i = 0
2  repeat j = h(k, i)
3     if T[j] == k
4        return j
5     i = i + 1
6  until T[j] == NIL or i == m
7  return NIL
```

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۲۸۰، با نمادگذاری *CLRS*

HASH-SEARCH-NEW در درایه‌ی deleted متوقف نمی‌شود، زیرا deleted با nil برابر نیست.

روش‌های وارسی

سه روش وارسی

«سه روش برای محاسبه‌ی دنباله‌ی وارسی به‌کار می‌رود. این روش‌ها تضمین می‌کنند که برای هر کلید k دنباله‌ی

$$\langle h(k,0), h(k,1), \ldots, h(k,m-1)\rangle$$

یک جای‌گشت از $\langle 0,1,\ldots,m-1\rangle$ باشد. البته هیچ‌یک از این روش‌ها، فرض درهم‌سازی یکنوا را ارضا نمی‌کند، چون هیچ‌کدام قادر به تولید بیش از m^2 دنباله‌ی وارسی نیست، که بسیار کم‌تر از $m!$ است. درهم‌سازی دوگانه بیش‌ترین تعداد دنباله‌ها را تولید می‌کند و گاهی از دو روش دیگر بهتر است.»

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۲۸۱

در درهم‌سازی یکنوا، دنباله‌ی وارسی هر کلید با احتمال برابر یکی از $m!$ جایگشت $\langle 0,1,\ldots,m-1\rangle$ است.

وارسی خطی

«با فرض داشتن یک تابع درهم‌سازی عادی $h' : U \to \{0, 1, \ldots, m-1\}$ که از آن به نام تابع درهم‌سازی کمکی (auxiliary) یاد می‌کنیم، روش «وارسی خطی» (linear probing) از تابع زیر برای $i = 0, 1, \ldots, m-1$ استفاده می‌کند:

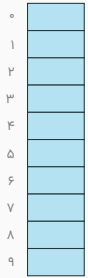
$$h(k, i) = (h'(k) + i) \bmod m \qquad (۴-۵)$$

به‌ازای یک کلید k ابتدا $T[h'(k)]$ را وارسی می‌کنیم، سپس به $T[h'(k)+1]$ و بعد به $T[h'(k)+2]$ می‌رویم و این کار را به‌صورت حلقوی ادامه می‌دهیم تا به $T[h'(k)-1]$ برسیم. روشن است که این روش فقط m دنباله‌ی وارسی مجزا تولید می‌کند.»

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۲۸۱

$m = ۱۰$ و $h'(k) = k \bmod ۱۰$. ترتیب درج: $۹۲, ۳۸, ۸۲, ۱۸, ۹۳, ۴۳, ۷۴$.

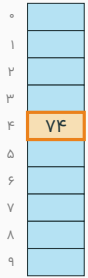
جدول خالی با ۱۰ درایه.



کلیدها و ترتیب درج از *CLRS, 4th ed.*، صفحه ۳۰۳، شکل ۶.۱۱

$m = ۱۰$ و $h'(k) = k \bmod ۱۰$. ترتیب درج: $۹۲, ۳۸, ۸۲, ۱۸, ۹۳, ۴۳, ۷۴$.

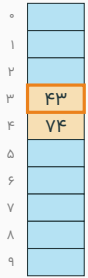
درج ۷۴: $h'(۷۴) = ۴$. یک وارسی.



کلیدها و ترتیب درج از *CLRS, 4th ed.*، صفحه ۳۰۳، شکل ۶.۱۱

$m = ۱۰$ و $h'(k) = k \bmod ۱۰$. ترتیب درج: $۹۲, ۳۸, ۸۲, ۱۸, ۹۳, ۴۳, ۷۴$.

درج ۴۳: $h'(۴۳) = ۳$. یک وارسی.



کلیدها و ترتیب درج از *CLRS, 4th ed.*، صفحه ۳۰۳، شکل ۶.۱۱

$m = ۱۰$ و $h'(k) = k \bmod ۱۰$. ترتیب درج: $۹۲, ۳۸, ۸۲, ۱۸, ۹۳, ۴۳, ۷۴$.

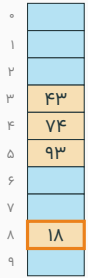
درج ۹۳: $h'(۹۳) = ۳$. درایه‌های ۳ و ۴ پرند و ۹۳ پس از ۳ وارسی در درایه‌ی ۵ قرار می‌گیرد.



کلیدها و ترتیب درج از *CLRS, 4th ed.*، صفحه ۳۰۳، شکل ۶.۱۱

$m = ۱۰$ و $h'(k) = k \bmod ۱۰$. ترتیب درج: $۹۲, ۳۸, ۸۲, ۱۸, ۹۳, ۴۳, ۷۴$.

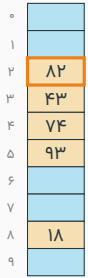
درج ۱۸: $h'(۱۸) = ۸$. یک وارسی.



کلیدها و ترتیب درج از *CLRS, 4th ed.*، صفحه ۳۰۳، شکل ۶.۱۱

$m = ۱۰$ و $h'(k) = k \bmod ۱۰$. ترتیب درج: $۷۴, ۴۳, ۹۳, ۱۸, ۸۲, ۳۸, ۹۲$.

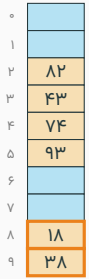
درج ۸۲: $h'(۸۲) = ۲$. یک وارسی.



کلیدها و ترتیب درج از *CLRS, 4th ed.*، صفحه ۳۰۳، شکل ۶.۱۱

$m = ۱۰$ و $h'(k) = k \bmod ۱۰$. ترتیب درج: $۹۲, ۳۸, ۸۲, ۱۸, ۹۳, ۴۳, ۷۴$.

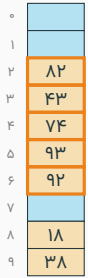
درج ۳۸: $h'(۳۸) = ۸$. درایه‌ی ۸ پر است و ۳۸ پس از ۲ وارسی در درایه‌ی ۹ قرار می‌گیرد.



کلیدها و ترتیب درج از *CLRS, 4th ed.*، صفحه ۳۰۳، شکل ۶.۱۱

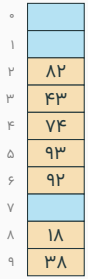
$m = ۱۰$ و $h'(k) = k \bmod ۱۰$. ترتیب درج: $۹۲, ۳۸, ۸۲, ۱۸, ۹۳, ۴۳, ۷۴$.

درج ۹۲: $h'(۹۲) = ۲$. درایه‌های ۲ تا ۵ پرند و ۹۲ پس از ۵ وارسی در درایه‌ی ۶ قرار می‌گیرد.



کلیدها و ترتیب درج از *CLRS, 4th ed.*، صفحه ۳۰۳، شکل ۶.۱۱

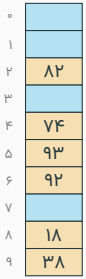
جدول اسلاید قبل، با وارسی خطی و $h'(k) = k \bmod ۱۰$:



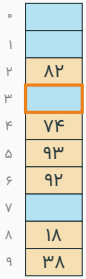
کلید ۹۳ هنگام درج، درایه‌های ۳ و ۴ را پر یافته و در درایه‌ی ۵ قرار گرفته است.

جدول اسلاید قبل، با وارسی خطی و $h'(k) = k \bmod ۱۰$:

$\textsc{Hash-Delete}(T, 43)$: درایه‌ی ۳ برابر nil می‌شود.



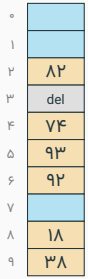
جدول اسلاید قبل، با وارسی خطی و $h'(k) = k \bmod ۱۰$:



HASH-SEARCH-NEW$(T, 93)$: $h(۹۳, ۰) = ۳$ و $T[۳] = \text{nil}$، پس جست‌وجو با پاسخ nil پایان می‌یابد، هرچند ۹۳ در درایه‌ی ۵ است.

ردیابی حذف و جست‌وجو

جدول اسلاید قبل، با وارسی خطی و $h'(k) = k \bmod ۱۰$:



به جای آن، $\text{HASH-DELETE-NEW}(T, 43)$ در درایه‌ی ۳ مقدار deleted می‌نویسد.

ردیابی حذف و جست‌وجو

جدول اسلاید قبل، با وارسی خطی و $h'(k) = k \bmod ۱۰$:



HASH-SEARCH-NEW$(T, 93)$: درایه‌ی ۳ برابر deleted است و جست‌وجو ادامه می‌یابد. ۹۳ پس از ۳ وارسی در درایه‌ی ۵ پیدا می‌شود.

جدول اسلاید قبل، با وارسی خطی و $h'(k) = k \bmod ۱۰$:

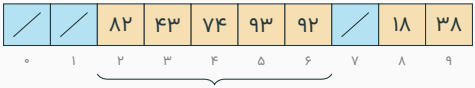


HASH-INSERT-NEW$(T, 13)$: HASH-SEARCH هر ۱۰ درایه را وارسی می‌کند و ۱۳ را نمی‌یابد. سپس از $h(۱۳, ۰) = ۳$ شروع می‌کنیم و چون $T[۳] = \text{deleted}$، کلید ۱۳ همان‌جا درج می‌شود.

خوشه‌بندی اولیه

«پیاده‌سازی وارسی خطی ساده است. ولی این روش مشکلی به‌نام «خوشه‌بندی اولیه» (primary clustering) دارد که موجب می‌شود تا در جدول، بخش‌های پر، دنبال هم ظاهر شوند که هر کدام را یک خوشه می‌گوییم. یک درایه‌ی خالی که بلافاصله بعد از i درایه‌ی پر دنبال هم قرار گیرد، در مرحله‌ی بعد و با احتمال $(i+1)/m$ پر می‌شود و به این ترتیب، به مرور زمان خوشه‌ها بزرگ‌تر شده و زمان جست‌وجو هم طولانی‌تر می‌شود.»

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۲۸۱



در جدول ردیابی، درایه‌های خالی ۰، ۱ و ۷ هستند. درایه‌ی ۷ پس از ۵ درایه‌ی پر با احتمال ۶/۱۰، درایه‌ی ۰ پس از ۲ درایه‌ی پر (۸ و ۹) با احتمال ۳/۱۰ و درایه‌ی ۱ با احتمال ۱/۱۰ پر می‌شود.

وارسی درجه‌ی ۲

««وارسی درجه‌ی ۲» (quadratic probing) از تابع درهم‌سازی زیر برای ($i = ۰, ۱, \ldots, m - ۱$) استفاده می‌کند:

$$h(k, i) = (h'(k) + c_۱ i + c_۲ i^۲) \bmod m \qquad (۵\text{-}۵)$$

که در آن h' تابع درهم‌سازی کمکی و $c_۱, c_۲$ ($c_۲ \neq ۰$) ثابت‌های کمکی هستند.»

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۲۸۲

«این روش بسیار بهتر از وارسی خطی عمل می‌کند و مشکل خوشه‌بندی را ندارد. البته برای این‌که همه‌ی m درایه‌ی جدول مورد استفاده قرار گیرند، مقادیر c_1، c_2 و m باید به‌صورت خاص انتخاب شوند. همچنین، اگر وارسی اولیه‌ی دو کلید مشابه باشند، دنباله‌ی وارسی‌های آن‌ها هم یک‌سان است

$$h(k_1, 0) = h(k_2, 0) \Longrightarrow h(k_1, i) = h(k_2, i).$$

»

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۲۸۲

مثال وارسی درجه‌ی ۲

دنباله‌ی وارسی کلیدی با $h'(k) = ۰$:

$m = ۱۰$، $c_۱ = ۰$، $c_۲ = ۱$

i	۰	۱	۲	۳	۴	۵	۶	۷	۸	۹
$h(k, i)$	۰	۱	۴	۹	۶	۵	۶	۹	۴	۱

$m = ۸$، $c_۱ = c_۲ = ۱/۲$

i	۰	۱	۲	۳	۴	۵	۶	۷
$h(k, i)$	۰	۱	۳	۶	۲	۷	۵	۴

در انتخاب اول فقط ۶ درایه از ۱۰ درایه وارسی می‌شود. در انتخاب دوم $h(k, i) = i(i+۱)/۲ \bmod ۸$ است و هر ۸ درایه وارسی می‌شود.

درهم‌سازی دوگانه

«درهم‌سازی دوگانه (double hashing) یک روش وارسی است که در آن تابع درهم‌سازی به‌صورت زیر است:

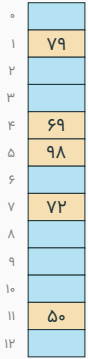
$$h(k, i) = (h_1(k) + i\, h_2(k)) \bmod m$$

که h_1 و h_2 تابع‌های درهم‌سازی کمکی هستند. اولین وارسی در $T[h_1(k)]$ انجام می‌شود و وارسی‌های بعدی به درایه‌هایی با فاصله‌ی $h_2(k)$ (البته به هنگ m) می‌رود.»

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۲۸۲

درج با درهم‌سازی دوگانه

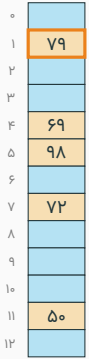
«در این مثال، $m = ۱۳$ و $h_۱(k) = k \bmod ۱۳$ و $h_۲(k) = ۱ + (k \bmod (۱۱$» کلید ۱۴ را درج می‌کنیم.



داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه‌های ۲۸۲ و ۲۸۳ (بازترسیم شکل ۵-۵)

درج با درهم‌سازی دوگانه

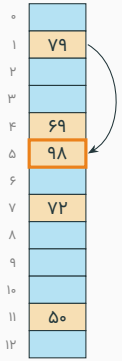
وارسی اول: $h_1(۱۴) = ۱$. درایه‌ی ۱ پر است.



داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه‌های ۲۸۲ و ۲۸۳ (بازترسیم شکل ۵-۵)

درج با درهم‌سازی دوگانه

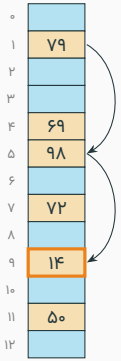
$h_۲(۱۴) = ۱ + ۳ = ۴$. وارسی دوم: $(۱ + ۴) \bmod ۱۳ = ۵$. درایه‌ی ۵ هم پر است.



داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه‌های ۲۸۲ و ۲۸۳ (بازترسیم شکل ۵-۵)

درج با درهم‌سازی دوگانه

«چون $۱۴ \equiv ۱ \pmod{۱۳}$ و $۱۴ \equiv ۳ \pmod{۱۱}$، بنابراین کلید ۱۴، پس از آن‌که درایه‌های ۱ و ۵ وارسی شد و پر بودند، در درایه‌ی خالی ۹ درج می‌شود.»



داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه‌های ۲۸۲ و ۲۸۳ (بازترسیم شکل ۵-۵)

«در طراحی این درهم‌سازی، باید مقدار $h_۲(k)$ نسبت به اندازه‌ی جدول، یا m اول باشد تا این‌که کلیه‌ی درایه‌های جدول وارسی شوند. یک روش مناسب برای اعمال این خاصیت، انتخاب m به‌صورت توان ۲ و طراحی $h_۲$ به‌طوری است که همیشه یک عدد فرد تولید کند. یک روش دیگر آن است که m عدد اول باشد و $h_۲$ همیشه یک عدد صحیح و مثبت کم‌تر از m را تولید کند.»

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه‌های ۲۸۲ و ۲۸۳

مثال

با $m = ۱۰$، $h_۱(k) = ۳$ و $h_۲(k) = ۵$ دنباله‌ی وارسی $۳, ۸, ۳, ۸, \ldots$ است و فقط ۲ درایه وارسی می‌شود. در حالت کلی تعداد درایه‌های وارسی‌شده $m/\gcd(h_۲(k), m)$ است.

یک انتخاب رایج

«مثلاً، m می‌تواند یک عدد اول و $h_۱$ و $h_۲$ به‌صورت زیر باشند:

$$h_۱(k) = k \bmod m, \qquad h_۲(k) = ۱ + (k \bmod m'),$$

که در آن m' عددی کوچک‌تر و نزدیک به m (مثلاً $m - ۱$) است. مثلاً اگر $k = ۱۲۳۴۵۶$ و $m = ۷۰۱$ و $m' = ۷۰۰$ باشد، داریم $h_۱(k) = ۸۰$ و $h_۲(k) = ۲۵۷$. بنابراین ابتدا درایه‌ی ۸۰ و سپس درایه‌های با فاصله‌ی ۲۵۷ از آن (به هنگ m) را وارسی می‌کنیم.»

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۲۸۳

چند وارسی بعدی: $h(k, ۱) = ۳۳۷$، $h(k, ۲) = ۵۹۴$ و $h(k, ۳) = (۸۰ + ۷۷۱) \bmod ۷۰۱ = ۱۵۰$.

مقایسه‌ی روش‌های وارسی

«درهم‌سازی دوگانه نسبت به وارسی خطی و وارسی درجه‌ی ۲ بهتر عمل می‌کند، چون به تعداد $\Theta(m^2)$ (در مقایسه با $\Theta(m)$) دنباله‌ی وارسی تولید می‌کند. بنابراین به‌نظر می‌رسد کارایی این درهم‌سازی به کارایی آرمانی درهم‌سازی یکنوا نزدیک باشد.»

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۲۸۳

درهم‌سازی سراسری

مشکل تابع ثابت

«اگر تابع درهم‌سازی را از قبل داشته باشیم، می‌توان مجموعه‌ای از n کلید را پیدا کرد که همه به یک درایه فرستاده شوند و در نتیجه زمان بازیافت عناصر $\Theta(n)$ گردد که بدترین حالت ممکن است. هر تابع درهم‌سازی مشخصی نیز ممکن است در بدترین حالت چنین رفتار بدی را از خود بروز دهد.»

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۲۷۲

مثال

با $h(k) = k \bmod ۱۰$، حریفی که h را می‌شناسد کلیدهای $۱۰, ۲۰, \ldots, ۱۰n$ را انتخاب می‌کند و همه در درایه‌ی ۰ قرار می‌گیرند. برای هر h ثابت چنین مجموعه‌ای وجود دارد، زیرا دست‌کم $\lceil |U|/m \rceil$ کلید از U به یک درایه می‌روند.

انتخاب تصادفی تابع

«تنها راه مؤثر برای بهبود وضعیت، انتخاب تصادفی تابع درهم‌سازی است به‌گونه‌ای که مستقل از مقادیری باشد که عملاً می‌خواهیم ذخیره کنیم. این روش را «درهم‌سازی سراسری» (universal hashing) می‌نامیم و نشان می‌دهیم که این روش بدون درنظر گرفتن چگونگی انتخاب مقادیر، در حالت میانگین عملکرد مناسبی خواهد داشت.»

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۲۷۲

ایده‌ی اصلی

«ایده‌ی اصلی درهم‌سازی سراسری این است که در شروع کار، تابع درهم‌سازی را از یک خانواده‌ی توابع که با دقت طراحی شده است انتخاب کنیم.»

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۲۷۲

$\mathcal{H}$ خانواده‌ای محدود از توابع $h : U \to \{0, 1, \ldots, m-1\}$ است.

«چنین خانواده‌ای را «سراسری» می‌گوییم اگر برای هر دو کلید متفاوت $k, l \in U$ تعداد توابع درهم‌سازی $h \in \mathcal{H}$ که برای آن‌ها $h(k) = h(l)$ است، حداکثر برابر $|\mathcal{H}|/m$ باشد. به بیان دیگر، اگر یک تابع درهم‌سازی h به صورت تصادفی از $\mathcal{H}$ انتخاب شود، احتمال وقوع برخورد بین کلیدهای متفاوت k و l با تابع h بیش‌تر از $1/m$ برابر شانس برخورد نیست، اگر مقدار $h(k)$ و $h(l)$ به‌صورت تصادفی و مستقل از هم از مجموعه‌ی $[0, 1, \ldots, m-1]$ انتخاب شده باشند.»

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۲۷۳

$$\Pr_{h \in \mathcal{H}} \{h(k) = h(l)\} = \frac{|\{h \in \mathcal{H} : h(k) = h(l)\}|}{|\mathcal{H}|} \leq \frac{1}{m}.$$

مثال خانواده‌ی سراسری

$U = \{۰, ۱, ۲\}$ و $m = ۲$. هر سطر یک تابع است:

	$h(۰)$	$h(۱)$	$h(۲)$
$h_۱$	۰	۰	۱
$h_۲$	۰	۱	۰
$h_۳$	۱	۰	۰

خانواده‌ی $\mathcal{H} = \{h_۱, h_۲, h_۳\}$ را بررسی می‌کنیم.
$|\mathcal{H}|/m = ۳/۲$.

مثال خانواده‌ی سراسری

$U = \{۰, ۱, ۲\}$ و $m = ۲$. هر سطر یک تابع است:

	$h(۰)$	$h(۱)$	$h(۲)$
$h_۱$	۰	۰	۱
$h_۲$	۰	۱	۰
$h_۳$	۱	۰	۰

- جفت $\{۰, ۱\}$ فقط با $h_۱$ برخورد دارد.
- جفت $\{۰, ۲\}$ فقط با $h_۲$ برخورد دارد.
- جفت $\{۱, ۲\}$ فقط با $h_۳$ برخورد دارد.

هر جفت با $۱ \leq ۳/۲$ تابع برخورد دارد، پس $\mathcal{H}$ سراسری است.

مثال خانواده‌ی سراسری

$U = \{۰, ۱, ۲\}$ و $m = ۲$. هر سطر یک تابع است:

	$h(۰)$	$h(۱)$	$h(۲)$
$h_۱$	۰	۰	۱
$h_۲$	۰	۱	۰
$h_۳$	۱	۰	۰
$h_۴$	۰	۰	۰

خانواده‌ی $\mathcal{H}' = \{h_۱, h_۴\}$ با $|\mathcal{H}'|/m = ۱$:

جفت $\{۰, ۱\}$ با هر دو تابع $h_۱$ و $h_۴$ برخورد دارد و $۱ < ۲$. پس $\mathcal{H}'$ سراسری نیست، هرچند جفت‌های دیگر با یک تابع برخورد دارند.

طول مورد انتظار لیست

«به‌خاطر بیاورید که n_i طول لیست $T[i]$ در درهم‌سازی به‌روش زنجیره‌ای است.»

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۲۷۳

قضیه‌ی ۵-۳

«فرض کنید تابع درهم‌سازی h از یک خانواده‌ی سراسری از توابع درهم‌سازی انتخاب شده و از آن برای درهم‌سازی n عنصر در جدول T با اندازه‌ی m استفاده شده است، و نیز برای حل مشکل برخوردها از روش زنجیره‌ای استفاده کرده‌ایم. اگر کلید k در جدول موجود نباشد، آن‌گاه $E[n_{h(k)}]$، یا امید ریاضی طول لیستی که k به آن فرستاده می‌شود، حداکثر برابر α است. اگر k در جدول موجود باشد، آن‌گاه $E[n_{h(k)}]$، یا امید ریاضی طول لیست شامل k، حداکثر برابر $1 + \alpha$ است.»

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۲۷۳

اثبات. «توجه کنید که امید ریاضی در این‌جا تنها وابسته به انتخاب تابع درهم‌سازی است و به فرضیات ما راجع به توزیع کلیدها ارتباطی ندارد.»

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۲۷۳

برای هر جفت کلید متفاوت k و l:

$$X_{kl} = I\{h(k) = h(l)\}, \qquad \Pr\{h(k) = h(l)\} \leq 1/m, \qquad E[X_{kl}] \leq 1/m.$$

Y_k تعداد برخوردهای کلید k با بقیه‌ی کلیدهای جدول است:

$$Y_k = \sum_{\substack{l \in T \\ l \neq k}} X_{lk}, \qquad E[Y_k] = E\Big[\sum_{\substack{l \in T \\ l \neq k}} X_{lk}\Big] = \sum_{\substack{l \in T \\ l \neq k}} E[X_{lk}] \leq \sum_{\substack{l \in T \\ l \neq k}} \frac{1}{m}.$$

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه‌های ۲۷۳ و ۲۷۴

دو حالت اثبات

«ادامه‌ی اثبات به این بستگی دارد که در جدول T مقدار k وجود دارد یا خیر.»

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۲۷۴

- $k \notin T$:

$$n_{h(k)} = Y_k, \quad |\{l : l \in T \text{ and } l \neq k\}| = n, \quad E[n_{h(k)}] = E[Y_k] \leq n/m = \alpha.$$

- $k \in T$: کلید k در لیست $T[h(k)]$ هست ولی Y_k آن را نمی‌شمارد.

$$n_{h(k)} = Y_k + ۱, \quad |\{l : l \in T \text{ and } l \neq k\}| = n - ۱,$$

$$E[n_{h(k)}] = E[Y_k] + ۱ \leq (n-۱)/m + ۱ = ۱ + \alpha - ۱/m < ۱ + \alpha.$$

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۲۷۴

مثال طول لیست

جدولی با $m = ۱۰۰۰$ درایه که $n = ۲۵۰۰$ کلید در آن با درهم‌سازی سراسری و زنجیره‌سازی درج شده است. پس $\alpha = ۲/۵$.

- جست‌وجوی کلیدی که در جدول نیست: امید طول لیست بررسی‌شده حداکثر $\alpha = ۲/۵$ است.
- جست‌وجوی کلیدی که در جدول هست: امید طول لیست آن حداکثر $۱ + \alpha - ۱/m = ۳/۴۹۹$ است.

مقایسه

با یک تابع ثابت، حریف می‌تواند هر ۲۵۰۰ کلید را در یک لیست قرار دهد. با درهم‌سازی سراسری این کران‌ها برای هر مجموعه‌ی کلید برقرارند، زیرا امید ریاضی فقط روی انتخاب h گرفته می‌شود.

حریف و تصادفی‌سازی

«نتیجه‌ی زیر نشان می‌دهد که حاصل درهم‌سازی سراسری مطلوب است. یعنی، با این روش برای حریف غیر ممکن است که دنباله‌ای از عملیات را انتخاب کند که سبب ایجاد بدترین حالت شود. اگر انتخاب تابع درهم‌سازی در هنگام اجرا به‌قدر کافی تصادفی باشد، تضمین می‌کنیم که می‌توان هر دنباله از اعمال را به‌طور میانگین در زمان اجرای مناسب انجام داد.»

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۲۷۴

نتیجه‌ی ۵-۱

«با استفاده از درهم‌سازی سراسری در جدولی با m درایه، اگر برخوردها با روش زنجیره‌ای حل شوند، هر دنباله از n عمل درج، جست‌وجو و حذف که $O(m)$ عمل درج را شامل شود، به زمان میانگین $\Theta(n)$ نیاز دارد.»

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۲۷۴

اثبات نتیجه‌ی ۵-۱. «چون تعداد عملیات درج $O(m)$ است، باید $n = O(m)$ باشد، بنابراین $\alpha = O(1)$. هر درج و حذف به زمان ثابت نیاز دارد، و طبق قضیه‌ی ۵-۳، میانگین زمان اجرای عمل جست‌وجو نیز $O(1)$ است. پس به دلیل خطی بودن فرضیات ما، امید ریاضی زمان اجرای کل عملیات $O(n)$ است.»

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه‌های ۲۷۴ و ۲۷۵

مثال

در جدولی با $m = ۱۰۰۰$ درایه، دنباله‌ای از $۱۰^۶$ عمل که حداکثر ۲۰۰۰ درج دارد اجرا می‌شود. در هر لحظه $\alpha \leq ۲$، پس امید زمان هر جست‌وجو حداکثر متناسب با $۱ + \alpha \leq ۳$ است و امید زمان کل دنباله $O(۱۰^۶)$ است.

ساخت خانواده‌ی سراسری

انتخاب عدد اول

«با کمک گرفتن از نظریه‌ی اعداد، می‌توان یک خانواده‌ی سراسری از توابع درهم‌سازی ساخت. ابتدا عدد اول p را به‌اندازه‌ی کافی بزرگ انتخاب می‌کنیم که تمامی مقادیر ممکن کلیدها (k) در بازه‌ی بسته‌ی $[0, p-1]$ قرار گیرند.»

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۲۷۵

$$Z_p = \{0, 1, \ldots, p-1\}, \qquad Z_p^* = \{1, 2, \ldots, p-1\}.$$

چون اندازه‌ی فضای کلیدها بزرگ‌تر از اندازه‌ی جدول فرض شده است، $p > m$.

مثال

برای کلیدهای ۰ تا ۹۹۹ و جدولی با $m = 100$ درایه می‌توان $p = 1009$ را انتخاب کرد، زیرا ۱۰۰۹ عددی اول و بزرگ‌تر از ۹۹۹ است.

تعریف توابع خانواده

«اکنون تابع درهم‌سازی $h_{a,b}$ را برای هر $a \in Z_p^*$ و هر $b \in Z_p$ به‌صورت زیر تعریف می‌کنیم:

$$h_{a,b}(k) = ((ak + b) \bmod p) \bmod m. \qquad (۵\text{-}۲)$$

به‌عنوان مثال، اگر $p = ۱۷$ و $m = ۶$ باشد، داریم $h_{۳,۴}(۸) = ۵$. خانواده‌ی تمامی این توابع درهم‌سازی مانند زیر است:

$$\mathcal{H}_{p,m} = \{h_{a,b} : a \in Z_p^* \text{ and } b \in Z_p\} \qquad (۵\text{-}۳)$$

»

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۲۷۵

«به‌دلیل این‌که $p - ۱$ انتخاب برای a و p انتخاب برای b داریم، $\mathcal{H}_{p,m}$ شامل $p(p - ۱)$ تابع می‌باشد.»

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۲۷۵

محاسبه‌ی مقدار هش

$p = ۱۷$، $m = ۶$ و تابع $h_{۳,۴}$. پس $|\mathcal{H}_{۱۷,۶}| = ۱۷ \cdot ۱۶ = ۲۷۲$.

k	۸	۲	۱۴	۶
$۳k + ۴$	۲۸			
$(۳k + ۴) \bmod ۱۷$	۱۱			
$h_{۳,۴}(k)$	۵			

برای $k = ۸$: ابتدا باقی‌مانده بر p و سپس باقی‌مانده بر m گرفته می‌شود.

محاسبه‌ی مقدار هش

$p = ۱۷$، $m = ۶$ و تابع $h_{۳,۴}$. پس $|\mathcal{H}_{۱۷,۶}| = ۱۷ \cdot ۱۶ = ۲۷۲$.

k	۸	۲	۱۴	۶
$۳k + ۴$	۲۸	۱۰	۴۶	
$(۳k + ۴) \bmod ۱۷$	۱۱	۱۰	۱۲	
$h_{۳,۴}(k)$	۵	۴	۰	

کلیدهای ۲ و ۱۴ به درایه‌های ۴ و ۰ می‌روند.

محاسبه‌ی مقدار هش

$p = ۱۷$، $m = ۶$ و تابع $h_{۳,۴}$. پس $|\mathcal{H}_{۱۷,۶}| = ۱۷ \cdot ۱۶ = ۲۷۲$.

k	۸	۱۴	۶
$۳k + ۴$	۲۸	۴۶	۲۲
$(۳k + ۴) \bmod ۱۷$	۱۱	۱۲	۵
$h_{۳,۴}(k)$	۵	۰	۵

برای برخورد با ۸ باید $(۳k + ۴) \bmod ۱۷ \in \{۵, ۱۱\}$ باشد. با $۳k + ۴ \equiv ۵ \pmod{۱۷}$ کلید $k = ۶$ به دست می‌آید. کلیدهای ۶ و ۸ پس از گرفتن باقی‌مانده بر ۱۷ متفاوت‌اند (۵ و ۱۱) و فقط در گرفتن باقی‌مانده بر ۶ برخورد می‌کنند.

قضیه‌ی ۵-۴

«خانواده‌ی $\mathcal{H}_{p,m}$ از توابع درهم‌سازی که توسط معادلات ۵-۲ و ۵-۳ تعریف شده است، سراسری است.»

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۲۷۵

اثبات. «دو مقدار مختلف k و l از Z_p را درنظر بگیرید. برای تابع درهم‌سازی داده‌شده‌ی $h_{a,b}$، r و s را مانند زیر تعریف می‌کنیم:

$$r = (ak + b) \bmod p, \qquad s = (al + b) \bmod p.$$

«

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه‌های ۲۷۵ و ۲۷۶

اثبات سه گام دارد: $r \neq s$، تناظر یک‌به‌یک میان (a, b) و (r, s)، و شمارش جفت‌هایی که $r \equiv s \pmod{m}$.

«مشاهده می‌کنیم که چون $r - s \equiv a(k - l) \pmod{p}$، p یک عدد اول است و a و $k - l$ به هنگ p ناصفر هستند، داریم $r \neq s$. بنابراین، هنگام محاسبه‌ی هر $h_{a,b}$ از $\mathcal{H}_{p,m}$، مقادیر متفاوت k و l به دو مقدار متفاوت r و s به هنگ p نگاشته می‌شوند. پس در مرحله‌ی گرفتن باقی‌مانده بر p برخوردی نداریم.»

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۲۷۶

مثال

در اسلاید محاسبه‌ی مقدار درهم، $k = ۶$ و $l = ۸$: $r - s = ۵ - ۱۱ \equiv ۱۱ \equiv ۳ \cdot (۶ - ۸) \pmod{۱۷}$ و $r \neq s$. برخورد فقط در گرفتن باقی‌مانده بر m رخ داد.

تناظر یک‌به‌یک

«به‌علاوه، هر یک از $p(p-1)$ انتخاب ممکن برای جفت (a,b) که $a \neq 0$، جفت (r,s) متفاوتی را نتیجه می‌دهد که $r \neq s$.» با داشتن r و s:

$$a = ((r-s)((k-l)^{-1} \bmod p)) \bmod p, \qquad b = (r - ak) \bmod p,$$

که $(k-l)^{-1} \bmod p$ وارون ضربی $k-l$ به هنگ p است.

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۲۷۶

چون دقیقاً $p(p-1)$ جفت (r,s) با $r \neq s$ وجود دارد، اگر (a,b) به‌صورت تصادفی و یکنواخت انتخاب شود، (r,s) با احتمال برابر هر یک از این جفت‌هاست.

مثال

$p = ۷$، $k = ۱$، $l = ۴$ و $(r,s) = (۰,۶)$. $(k-l)^{-1} \equiv ۴^{-1} \equiv ۲ \pmod ۷$، پس $a = (-۶ \cdot ۲) \bmod ۷ = ۲$ و $b = (۰ - ۲) \bmod ۷ = ۵$. بررسی: $h_{۲,۵}$ برای $k = ۱$ مقدار $r = ۷ \bmod ۷ = ۰$ و برای $l = ۴$ مقدار $s = ۱۳ \bmod ۷ = ۶$ را می‌دهد.

شمارش برخوردها

«برای مقدار داده‌شده‌ی r، تعداد مقادیر ممکن برای s از بین $۱ - p$ مقدار باقی‌مانده که $s \neq r$ و $s \equiv r \pmod{p}$ حداکثر برابر زیر است:

$$\lceil p/m \rceil - ۱ \leq (p + m - ۱)/m - ۱ = (p - ۱)/m.$$

»

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۲۷۶

پس برای هر r، احتمال برخورد حداکثر $\frac{(p-۱)/m}{p-۱} = \frac{۱}{m}$ است و برای هر دو مقدار متفاوت $k, l \in Z_p$:

$$\Pr\{h_{a,b}(k) = h_{a,b}(l)\} \leq ۱/m.$$

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۲۷۶

مثال شمارش برخوردها

$p = ۷$ و $m = ۳$، پس $|\mathcal{H}_{۷,۳}| = ۴۲$ و $|\mathcal{H}|/m = ۱۴$. کلیدها $k = ۱$ و $l = ۴$.

باقی‌مانده بر ۳	مقادیر $Z_۷$
۰	۰, ۳, ۶
۱	۱, ۴
۲	۲, ۵

برخورد یعنی $r \neq s$ و $r \equiv s \pmod ۳$. پس r و s دو عضو متفاوت از یک سطر جدول‌اند.

مثال شمارش برخوردها

$p = ۷$ و $m = ۳$، پس $|\mathcal{H}_{۷,۳}| = ۴۲$ و $|\mathcal{H}|/m = ۱۴$. کلیدها $k = ۱$ و $l = ۴$.

باقی‌مانده بر ۳	مقادیر $Z_۷$
۰	۰, ۳, ۶
۱	۱, ۴
۲	۲, ۵

تعداد جفت‌های مرتب (r, s):

$$۳ \cdot ۲ + ۲ \cdot ۱ + ۲ \cdot ۱ = ۱۰.$$

برای $r = ۰$ دو مقدار ممکن برای s داریم که برابر کران $(p - ۱)/m = ۲$ است.

مثال شمارش برخوردها

$p = ۷$ و $m = ۳$، پس $|\mathcal{H}_{۷,۳}| = ۴۲$ و $|\mathcal{H}|/m = ۱۴$. کلیدها $k = ۱$ و $l = ۴$.

باقی‌مانده بر ۳	مقادیر $Z_۷$
۰	۰, ۳, ۶
۱	۱, ۴
۲	۲, ۵

با تناظر یک‌به‌یک، دقیقاً ۱۰ تابع از ۴۲ تابع کلیدهای ۱ و ۴ را برخورد می‌دهند. یکی از آن‌ها $h_{۲,۵}$ است. چون $۱۴ \leq ۱۰$، احتمال برخورد $۱/۳ \leq ۰/۲۴ \approx ۱۰/۴۲$ است.

منابع

منابع

- محمد قدسی، داده‌ساختارها و مبانی الگوریتم‌ها، انتشارات فاطمی، چاپ ۱۳۹۷
- T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, *Introduction to Algorithms*, 4th ed., MIT Press, 2022.